How to Use: Just click through the levels on the website!

Introduction:

The purpose of this project was to get familiar with WebGL and GLSL. This was done by implementing four separate levels that build off each other:

Level 1:



Fig 1. Level One Implementation

The goal of level one was to create a procedural landscape on a certain plane (x-z, y-z, .. etc.) using only cube objects. I took a bit of a twist and transformed these cubes using trigonometric functions to get a neat landscape. Using a pretty standard fragment and vertex shader, we can transform the coordinates of our cubes to the world and use the full brightness of the color to get a whole lot of cubes. This helped me get an understanding of what procedural landscapes need (a whole lot of noise!) to feel unique.

Level 2:

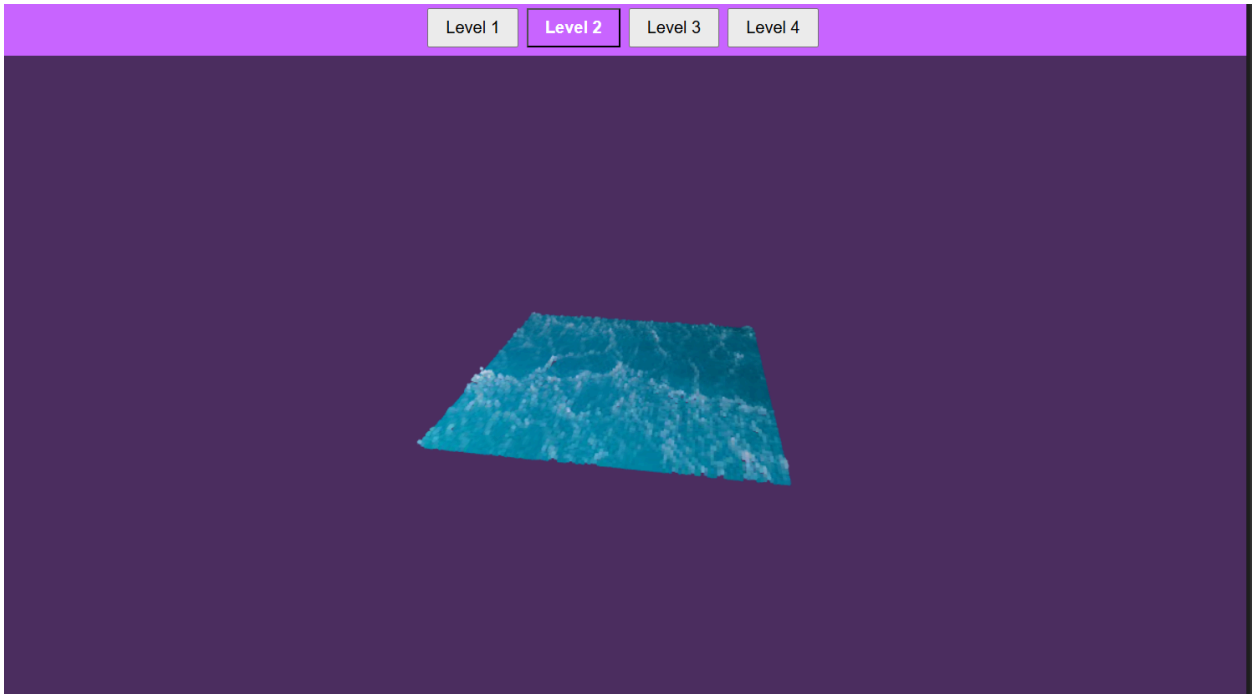


Fig 2. Level Two Implementation

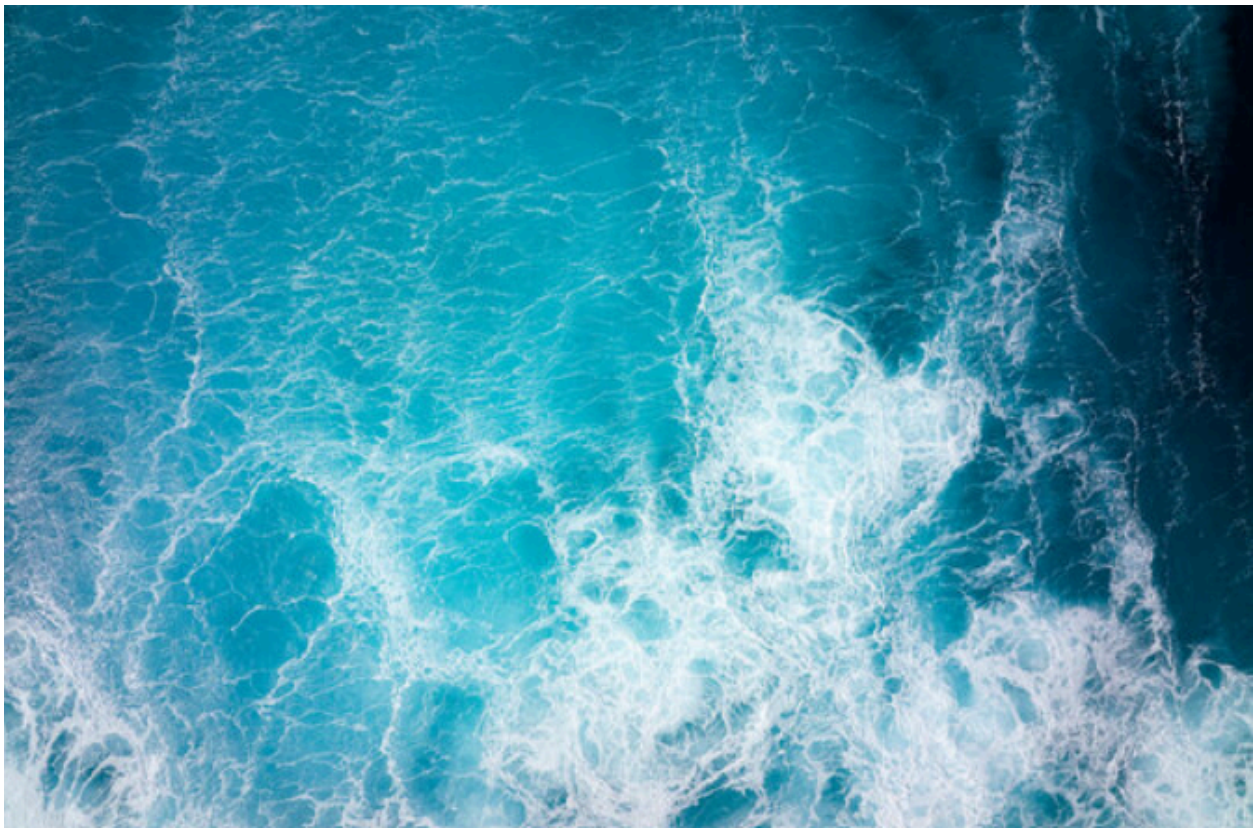


Fig 3. Bitmap Image

The goal of level two was to create a cube landscape using an image. Using this image, we can just take the ‘whiteness’ of the value and map it to the heights of each cube. Sampling was used in order to achieve the landscape (this image has way too high a fidelity!). This level allowed me to implement coordinate mapping in a visual way! As you can probably see, due to the sampling (this grid is only 200 x 200), it has more of a pixelated look which I love. It kind of gives the effect of texturing without actually applying textures.

Level 3:

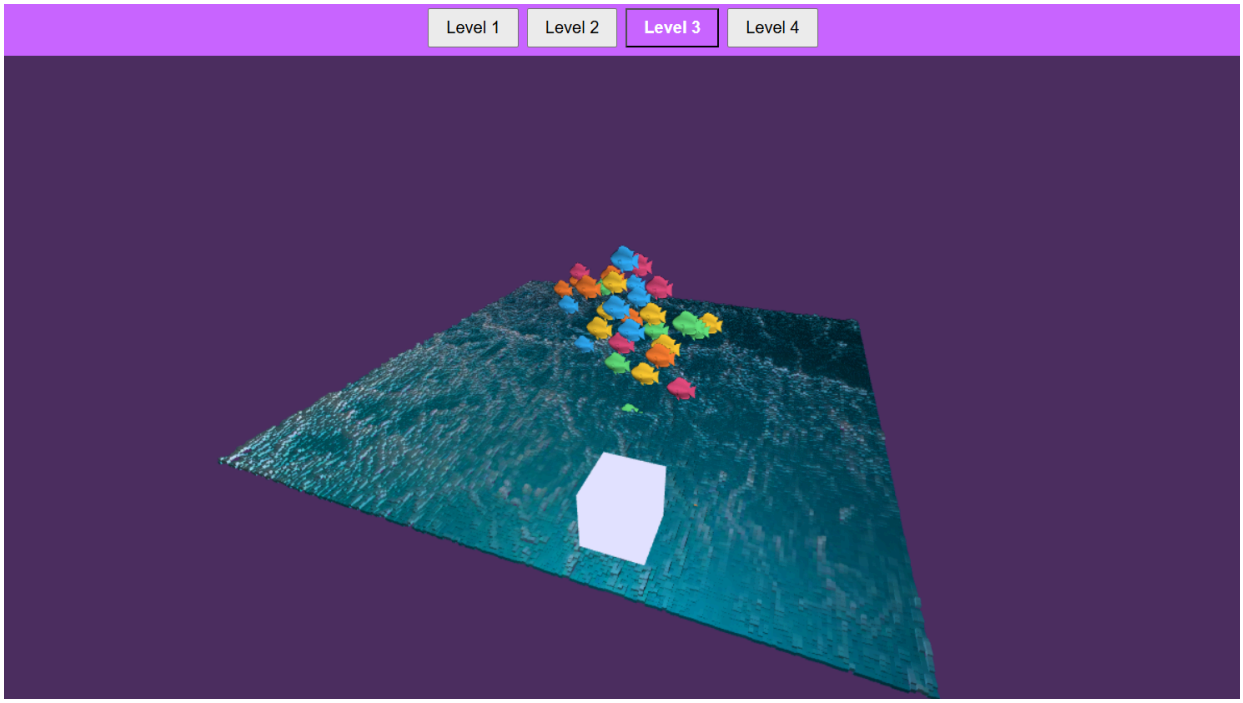


Fig 4. Level Three Implementation

The goal of level three was to add bezier curves and objects in the sky! Here I decided to add fishes and a moon. A bezier curve was used for the fish's motion and each one has an offset to achieve a scattered (but unified) look. The moon was actually a little trickier to implement for me because of the new and improved lighting added. Easiest way for me to understand what’s going on here is by imagining a light vector pointing straight out of the moon and changing up the color on the fragments each frame (by using angle of the light vector and the normal of the fragment).

Level 4:

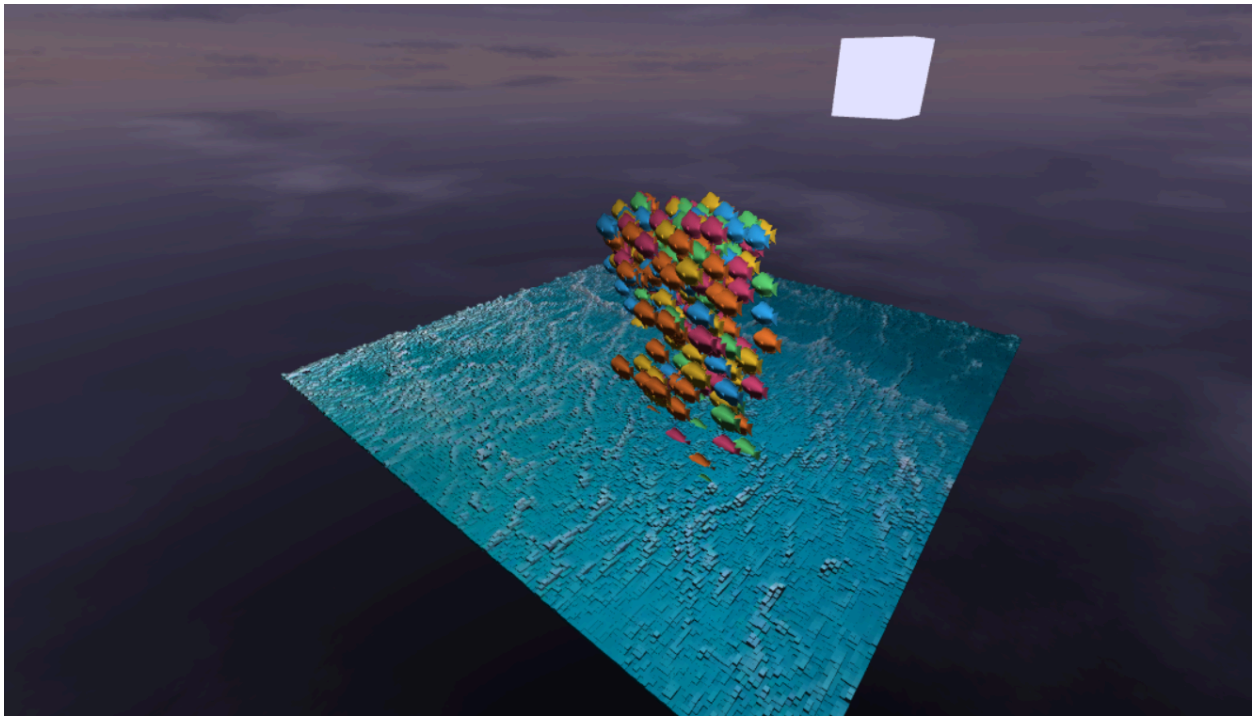


Fig 5. Level 4 Implementation

The goal of the final part was to experiment! Since I already added lighting, I decided to add a skybox, which I did not expect to be actually laid out like a box in the image file. Implementing skyboxes got… interesting. There were some precautions I had to specify in the program so that the cubemap actually wraps around and does not reach the black spaces in the image file. New vertex and fragment shaders were created as well, because if we used the ones used for the rest of the object, we probably would run into a whole lot of issues. This is because we do not want the camera's adjustments to impact how the sky looks in the scene.

What to do next:

I want to implement motion in my cubemap. This would be a fun challenge and require a bit of optimization because of the need for more cubes. I would like to implement some more movement and interactive components to Cube World as well.